DOCUMENTACIÓN DEL PROYECTO

Sistema de Gestión de Reservas

Agencia de Viajes Multi-Sucursal

Fabiana Ospina Londoño
Sebastián Salazar Vargas

Arquitectura MVC | Python 3 | Programación Orientada a Objetos

2026

1. Que es este proyecto y para que sirve

Este sistema fue construido para una agencia de viajes con varias sucursales en el pais. Antes de este sistema, cada sucursal llevaba sus reservas en hojas de calculo separadas, lo que causaba problemas como reservas duplicadas, errores en disponibilidad y perdida de informacion de turistas.

El sistema permite gestionar de forma centralizada y en memoria (sin base de datos) los siguientes elementos:

Elemento	Para que sirve
Turista	Persona que viaja y hace reservas de hoteles o vuelos.
Sucursal	Oficina de la agencia que vende y gestiona las reservas.
Hotel	Lugar de hospedaje con un numero limitado de plazas.
Vuelo	Trayecto aereo con un numero limitado de cupos.
Reserva	Vinculo entre un turista, un hotel o vuelo, y una sucursal.

2. Como esta organizado el proyecto (Arquitectura MVC)

El proyecto usa la arquitectura MVC (Modelo - Vista - Controlador). Esto significa que el codigo esta dividido en tres capas con responsabilidades claras y separadas. Piensalo como una cocina de restaurante:

Capa	Carpeta	Analogia	Responsabilidad
Modelo	app/models/	Los ingredientes y recetas	Guarda los datos y las reglas del negocio.
Controlador	app/controllers/	El chef que cocina	Coordina modelos, valida y ejecuta acciones.
Vista	app/main.py	El mesero que sirve	Muestra resultados al usuario por consola.

2.1 Estructura de carpetas

```
app/
  models/
    hotel.py          -> Clase Hotel (plazas de hospedaje)
    vuelo.py          -> Clase Vuelo (cupos aereos)
    turista.py        -> Clase Turista (viajero)
    sucursal.py       -> Clase Sucursal (oficina de ventas)
    reserva.py        -> Clase abstracta Reserva (molde base)
    reservaHotel.py   -> Reserva especifica de hotel
    reservaVuelo.py   -> Reserva especifica de vuelo
  controllers/
    hotel.py          -> Logica de negocio para hoteles
    vuelo.py          -> Logica de negocio para vuelos
    turista.py        -> Logica de negocio para turistas
    sucursal.py       -> Logica de negocio para sucursales
    reserva.py        -> Logica de negocio para reservas
  main.py             -> Vista: punto de entrada del sistema
  __init__.py         -> Archivo vacio, necesario para Python
```

3. Los Modelos (app/models/)

Los modelos son las clases que representan los objetos del mundo real. Cada modelo guarda sus datos de forma privada (encapsulamiento) y solo los expone a traves de metodos publicos llamados getters.

> REGLA DE ORO: Nunca accedas directamente a los atributos de un modelo.
> MAL: hotel.__plazas_disponibles
> BIEN: hotel.get_plazas()

3.1 Hotel (models/hotel.py)

Representa un hotel con un numero limitado de plazas de hospedaje.

```
class Hotel:
    def __init__(self, id_hotel: int, nombre: str, plazas_disponibles: int)

    get_id()            -> Retorna el id del hotel
    get_nombre()        -> Retorna el nombre del hotel
    get_plazas()        -> Retorna cuantas plazas quedan libres
    descontar_plaza()   -> Resta 1 plaza. Retorna True si habia cupo, False si
no
    liberar_plaza()     -> Suma 1 plaza (cuando se cancela una reserva)
```

Ejemplo de uso:

```
hotel = Hotel(1, 'Hotel Andino', 5)
print(hotel.get_plazas())    # Imprime: 5
hotel.descontar_plaza()      # Ahora quedan 4 plazas
hotel.liberar_plaza()        # Ahora quedan 5 plazas de nuevo
```

3.2 Vuelo (models/vuelo.py)

Representa un vuelo con un numero limitado de cupos disponibles. Funciona identico a Hotel pero para aviones.

```
class Vuelo:
    def __init__(self, id_vuelo: int, aerolinea: str, cupos: int)

    get_id()            -> Retorna el id del vuelo
    get_aerolinea()     -> Retorna el nombre de la aerolinea
    get_cupos()         -> Retorna cuantos cupos quedan
    descontar_cupo()    -> Resta 1 cupo. Retorna True si habia cupo, False si no
    liberar_cupo()      -> Suma 1 cupo (cuando se cancela una reserva)
```

3.3 Turista (models/turista.py)

Representa a la persona que viaja. Solo almacena informacion de contacto.

```
class Turista:
    def __init__(self, id_turista: int, nombre: str, telefono: str)

    get_id()         -> Retorna el id del turista
    get_nombre()     -> Retorna el nombre completo
    get_telefono()   -> Retorna el telefono de contacto
```

3.4 Sucursal (models/sucursal.py)

Representa una oficina de la agencia. IMPORTANTE: La sucursal tiene una lista interna de reservas. Esto se llama COMPOSICION: las reservas nacen y mueren dentro de la sucursal. Si eliminas la sucursal, sus reservas desaparecen con ella.

```
class Sucursal:
    def __init__(self, id_sucursal: int, nombre: str, direccion: str)
    # Internamente crea: self.__lista_reservas = []  <- COMPOSICION

    get_id()                -> Retorna el id de la sucursal
    get_nombre()            -> Retorna el nombre de la sucursal
    get_direccion()         -> Retorna la direccion fisica
    agregar_reserva(r)      -> Agrega una reserva a la lista interna
    get_reservas()          -> Retorna toda la lista de reservas
    get_total_reservas()    -> Retorna cuantas reservas tiene
```

3.5 Reserva (models/reserva.py) - Clase Abstracta

Esta es la clase BASE de todas las reservas. Es abstracta, lo que significa que NO puedes crear un objeto Reserva directamente. Es solo un molde que obliga a sus hijos (ReservaHotel y ReservaVuelo) a implementar el metodo validar_disponibilidad().

> QUE ES UNA CLASE ABSTRACTA?
> Es como un contrato. Dice: 'todo el que herede de mi DEBE implementar estos metodos'.
> Si intentas crear Reserva() directamente, Python lanzara un error.

```
from abc import ABC, abstractmethod

class Reserva(ABC):
    def __init__(self, id_reserva, fecha, turista)
    # Atributos con _ (protegidos, accesibles por los hijos):
    # _id_reserva, _fecha, _turista, _estado = 'Pendiente'

    validar_disponibilidad()  -> ABSTRACTO: los hijos lo implementan
    get_id()                  -> Retorna el id de la reserva
    get_estado()              -> Retorna: Pendiente / Confirmada / Cancelada
    get_turista()             -> Retorna el objeto Turista asociado
    get_fecha()               -> Retorna la fecha de la reserva
```

3.6 ReservaHotel (models/reservaHotel.py)

Hereda de Reserva y agrega la logica especifica para reservas de hotel. Contiene una referencia a un objeto Hotel (esto se llama AGREGACION: el hotel existe independientemente de la reserva).

```
class ReservaHotel(Reserva):
    def __init__(self, id_reserva, fecha, turista, hotel)
    # self.__hotel = hotel  <- AGREGACION

    get_hotel()               -> Retorna el objeto Hotel asociado
    validar_disponibilidad()  -> True si hotel.get_plazas() > 0
    confirmar()               -> Descuenta plaza y cambia estado a Confirmada
                                 Retorna True si exitoso, False si sin cupo
    cancelar()                -> Libera plaza y cambia estado a Cancelada
                                 Solo funciona si el estado es Confirmada
```

3.7 ReservaVuelo (models/reservaVuelo.py)

Identica a ReservaHotel pero para vuelos. Contiene una referencia a un objeto Vuelo.

```
class ReservaVuelo(Reserva):
    def __init__(self, id_reserva, fecha, turista, vuelo)
    # self.__vuelo = vuelo  <- AGREGACION
```

```
    get_vuelo()                -> Retorna el objeto Vuelo asociado
    validar_disponibilidad()   -> True si vuelo.get_cupos() > 0
    confirmar()                -> Descuenta cupo y cambia estado a Confirmada
    cancelar()                 -> Libera cupo y cambia estado a Cancelada
```

4. Los Controladores (app/controllers/)

Los controladores son los intermediarios entre los modelos y la vista. Cada controlador maneja una lista en memoria de sus objetos, valida las entradas y coordina las acciones entre modelos.

> REGLA: El controlador NUNCA accede directamente a los atributos privados de un modelo.
> Solo usa los metodos publicos (getters y metodos de accion).
> La vista NUNCA manipula modelos directamente, siempre pasa por el controlador.

4.1 HotelController (controllers/hotel.py)

Gestiona la lista de hoteles en memoria. Es el unico lugar donde se crean y buscan hoteles.

Metodo	Que hace
registrar_hotel(id, nombre, plazas)	Crea un Hotel y lo guarda en la lista. Valida que el id no exista y que las plazas no sean negativas.
consultar_hoteles()	Retorna la lista completa de hoteles registrados.
consultar_disponibilidad(id_hotel)	Busca el hotel por id e imprime cuantas plazas tiene disponibles.
actualizar_nombre(id_hotel, nuevo_nombre)	Actualiza el nombre de un hotel existente.
__buscar_por_id(id_hotel)	PRIVADO. Busca internamente un hotel por su id. No lo llames desde afuera.

4.2 VueloController (controllers/vuelo.py)

Identico a HotelController pero para vuelos.

Metodo	Que hace
registrar_vuelo(id, aerolinea, cupos)	Crea un Vuelo y lo guarda. Valida id unico, aerolinea no vacia y cupos positivos.
consultar_vuelos()	Retorna la lista completa de vuelos.
consultar_vuelo(id_vuelo)	Retorna un vuelo especifico por id.
consultar_disponibilidad(id_vuelo)	Imprime cuantos cupos quedan en el vuelo.
actualizar_aerolinea(id, nueva_aerolinea)	Actualiza el nombre de la aerolinea.

4.3 TuristaController (controllers/turista.py)

Metodo	Que hace
registrar_turista(id, nombre, telefono)	Crea un Turista. Valida que el id no exista y que nombre y telefono no esten vacios.
consultar_turistas()	Retorna la lista completa de turistas.
consultar_turista(id_turista)	Retorna un turista especifico por id.
actualizar_telefono(id, nuevo_telefono)	Actualiza el telefono de un turista.
buscar_por_nombre(nombre)	Busca turistas cuyo nombre contenga el texto ingresado (busqueda parcial).

4.4 SucursalController (controllers/sucursal.py)

Metodo	Que hace
registrar_sucursal(id, nombre, direccion)	Crea una Sucursal. Valida id unico y que nombre y direccion no esten vacios.
consultar_sucursales()	Retorna la lista completa de sucursales.
consultar_sucursal(id_sucursal)	Retorna una sucursal especifica por id.
actualizar_nombre(id, nuevo_nombre)	Actualiza el nombre de la sucursal.
consultar_reservas_sucursal(id)	Muestra cuantas reservas tiene registradas una sucursal (trazabilidad).

4.5 ReservaController (controllers/reserva.py)

Este es el controlador mas importante del sistema. Coordina la creacion de reservas entre turistas, hoteles o vuelos, y sucursales. Tambien genera los IDs de reserva automaticamente.

Metodo	Que hace
crear_reserva_hotel(turista, hotel, sucursal, fecha)	Crea una ReservaHotel, la confirma, la agrega a la sucursal y a la lista interna. Si no hay plazas, retorna None.
crear_reserva_vuelo(turista, vuelo, sucursal, fecha)	Igual que el anterior pero para vuelos.
cancelar_reserva(id_reserva)	Busca la reserva por id, valida que este Confirmada y llama su metodo cancelar().
consultar_reservas()	Retorna la lista de todas las reservas del sistema.
consultar_por_turista(id_turista)	Filtra y retorna solo las reservas de un turista especifico.
__generar_id()	PRIVADO. Genera un id unico autoincremental para cada nueva reserva.

5. Como se conectan las piezas (Flujos principales)

5.1 Flujo: Crear una reserva de hotel exitosa

```
# 1. La vista llama al controlador con los objetos necesarios
reserva = reserva_ctrl.crear_reserva_hotel(turista_1, hotel_1, sucursal_bog,
fecha)

# 2. ReservaController crea el objeto ReservaHotel
reserva = ReservaHotel(id_generado, fecha, turista, hotel)

# 3. Llama confirmar() en el modelo ReservaHotel
reserva.confirmar()
  -> validar_disponibilidad()  # hotel.get_plazas() > 0 ?
  -> hotel.descontar_plaza()   # resta 1 plaza al hotel
  -> self._estado = 'Confirmada'

# 4. Agrega la reserva a la sucursal (trazabilidad)
sucursal.agregar_reserva(reserva)

# 5. Retorna el objeto reserva a la vista
return reserva  # estado: Confirmada
```

5.2 Flujo: Crear una reserva sin disponibilidad

```
# Hotel con 0 plazas
reserva = reserva_ctrl.crear_reserva_hotel(turista, hotel_sin_plazas, sucursal,
fecha)

# confirmar() llama validar_disponibilidad()
# hotel.get_plazas() retorna 0 -> False
# self._estado = 'Rechazada'
# confirmar() retorna False

# El controlador imprime el error y retorna None
return None
```

5.3 Flujo: Cancelar una reserva

```
reserva_ctrl.cancelar_reserva(1)  # id de la reserva a cancelar

# 1. Busca la reserva por id en la lista interna
reserva = __buscar_por_id(1)

# 2. Valida que este Confirmada
reserva.get_estado() == 'Confirmada'  # True -> continua
```

```
# 3. Llama cancelar() en el modelo
reserva.cancelar()
  -> hotel.liberar_plaza()   # devuelve 1 plaza al hotel
  -> self._estado = 'Cancelada'
```

6. Como agregar nuevas funcionalidades

6.1 Agregar un nuevo campo a un modelo existente

Ejemplo: agregar el campo 'email' a Turista.

```
# En models/turista.py
class Turista:
    def __init__(self, id_turista: int, nombre: str, telefono: str, email:
str):
        self.__id_turista = id_turista
        self.__nombre = nombre
        self.__telefono = telefono
        self.__email = email  # <- nuevo campo

    def get_email(self) -> str:  # <- nuevo getter
        return self.__email
```

Luego actualiza el controlador para recibir el nuevo parametro:

```
# En controllers/turista.py
def registrar_turista(self, id_turista, nombre, telefono, email):  # <- agrega
email
    ...
    turista = Turista(id_turista, nombre, telefono, email)  # <- pasa email
```

6.2 Agregar un nuevo tipo de reserva

Ejemplo: agregar ReservaTraslado (reserva de transporte terrestre).

```
# 1. Crea models/reservaTraslado.py
from .reserva import Reserva

class ReservaTraslado(Reserva):
    def __init__(self, id_reserva, fecha, turista, traslado):
        super().__init__(id_reserva, fecha, turista)
        self.__traslado = traslado

    def validar_disponibilidad(self) -> bool:
        return self.__traslado.get_cupos() > 0  # logica propia

    def confirmar(self) -> bool:
        if self.validar_disponibilidad():
            self.__traslado.descontar_cupo()
            self._estado = 'Confirmada'
            return True
        return False

# 2. Agrega crear_reserva_traslado() en controllers/reserva.py
# 3. Llama el nuevo metodo desde main.py
```

6.3 Agregar un nuevo controlador

Sigue siempre este patron base:

```
from models.mi_modelo import MiModelo

class MiModeloController:
    def __init__(self):
        self.__lista = []  # lista en memoria

    def registrar(self, id, dato1, dato2) -> MiModelo:
        if self.__buscar_por_id(id):  # validar duplicado
            print('Error: ya existe.')
            return None
        obj = MiModelo(id, dato1, dato2)
        self.__lista.append(obj)
        return obj

    def __buscar_por_id(self, id) -> MiModelo:  # metodo privado
        for obj in self.__lista:
            if obj.get_id() == id:
                return obj
        return None
```

7. Errores comunes y como solucionarlos

Error	Causa	Solucion
TypeError: X() takes no arguments	El constructor tiene _init_ con un solo guion bajo.	Cambiar a __init__ (doble guion bajo).
AttributeError: object has no attribute 'get_X'	Falta el getter en el modelo o el nombre esta mal escrito.	Agregar el metodo get_X() al modelo correspondiente.
ModuleNotFoundError: No module named 'models'	Ejecutas main.py desde la carpeta equivocada.	Ir a la carpeta app/ con cd app y ejecutar python main.py.
NameError: name 'void' is not defined	void no existe en Python.	Reemplazar -> void por -> None en todos los metodos.
Reserva retorna None inesperadamente	El hotel o vuelo no tiene disponibilidad.	Verificar hotel.get_plazas() o vuelo.get_cupos() antes de reservar.
Error al cancelar: reserva no confirmada	Se intenta cancelar una reserva que ya fue cancelada o rechazada.	Verificar reserva.get_estado() antes de cancelar.

8. Glosario de terminos

Termino	Que significa en este proyecto
Encapsulamiento	Los atributos de un modelo son privados (self.__atributo). Solo se acceden con getters.
Getter	Metodo publico que retorna el valor de un atributo privado. Ej: get_nombre().
Clase Abstracta (ABC)	Clase que no se puede instanciar directamente. Define metodos que sus hijos deben implementar.
Herencia	ReservaHotel hereda de Reserva. Recibe todos sus atributos y metodos automaticamente.
Composicion	Sucursal tiene una lista de Reservas. Si la sucursal desaparece, las reservas tambien.
Agregacion	ReservaHotel tiene un Hotel. El hotel existe aunque no haya reservas.
super().__init__()	Llama al constructor de la clase padre. ReservaHotel lo usa para inicializar los campos de Reserva.
MVC	Patron de arquitectura: Modelo (datos), Vista (consola), Controlador (logica).
En memoria	Los datos se guardan en listas de Python. Al cerrar el programa, los datos se pierden.